

Proyecto Final — JPA & EntityManager

Sistema de Gestión de Reservas de Restaurante

Módulo: Programación · Tema 8 – Persistencia de datos con JPA y Hibernate

Entrega: Repositorio GitHub (enlace a través del aula virtual)

Contexto

Una cadena de restaurantes necesita una aplicación de consola para gestionar sus **restaurantes**, **mesas**, **clientes** y **reservas**. Hasta ahora utilizaban hojas de cálculo, pero quieren migrar el sistema a una **base de datos MySQL** usando **Jakarta Persistence (JPA)** con **Hibernate** como proveedor.

El objetivo del proyecto es practicar el uso de `EntityManager`, `EntityManagerFactory`, el patrón **Repository** y consultas **JPQL**. El programa no tendrá interfaz gráfica ni web: se ejecutará desde consola.

Estructura del proyecto Maven

```
proyecto-jpa-restaurantes/  
└─ src/  
    └─ main/  
        ├── resources/  
        │   └─ META-INF/  
        │       └─ persistence.xml  
        └─ java/  
            └─ com/  
                └─ restaurantes/  
                    ├── Main.java  
                    ├── modelo/  
                    │   ├── Restaurante.java  
                    │   ├── Mesa.java  
                    │   ├── Cliente.java  
                    │   ├── Reserva.java  
                    │   └─ EstadoReserva.java  
                    ├── repositorio/  
                    │   ├── Repository.java  
                    │   ├── JpaRepository.java  
                    │   ├── RestauranteRepository.java  
                    │   ├── MesaRepository.java  
                    │   ├── ClienteRepository.java  
                    │   └─ ReservaRepository.java  
                    ├── servicio/  
                    │   └─ ReservaServicio.java  
                    └─ util/  
                        └─ JpaUtil.java
```

Organiza el código exactamente en estos paquetes. No se valorará un proyecto que no compile o que mezcle clases en el paquete raíz.

Dependencias Maven

Incluye en tu `pom.xml` las siguientes dependencias:

```
<!-- Hibernate ORM como proveedor JPA -->
<dependency>
  <groupId>org.hibernate.orm</groupId>
  <artifactId>hibernate-core</artifactId>
  <version>7.0.0.Final</version>
</dependency>

<!-- Conector MySQL -->
<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <version>9.2.0</version>
</dependency>

<!-- Jakarta Persistence API -->
<dependency>
  <groupId>jakarta.persistence</groupId>
  <artifactId>jakarta.persistence-api</artifactId>
  <version>3.2.0</version>
</dependency>

<!-- Lombok -->
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
  <version>1.18.34</version>
  <scope>provided</scope>
</dependency>
```

Configuración de JPA

Crea el fichero `src/main/resources/META-INF/persistence.xml` :

```

<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="https://jakarta.ee/xml/ns/persistence"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="https://jakarta.ee/xml/ns/persistence

https://jakarta.ee/xml/ns/persistence/persistence_3_2.xsd"
    version="3.2">

    <persistence-unit name="restaurantesPU" transaction-type="RESOURCE_LOCAL">
        <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>

        <class>com.restaurantes.modelo.Restaurante</class>
        <class>com.restaurantes.modelo.Mesa</class>
        <class>com.restaurantes.modelo.Cliente</class>
        <class>com.restaurantes.modelo.Reserva</class>
        <class>com.restaurantes.modelo.EstadoReserva</class>

        <properties>
            <property name="jakarta.persistence.jdbc.driver"
value="com.mysql.cj.jdbc.Driver"/>
            <property name="jakarta.persistence.jdbc.url"
value="jdbc:mysql://localhost:3306/restaurantes_db"/>
            <property name="jakarta.persistence.jdbc.user" value="TU_USUARIO"/>
            <property name="jakarta.persistence.jdbc.password" value="TU_PASSWORD"/>

            <property name="hibernate.dialect"
value="org.hibernate.dialect.MySQLDialect"/>
            <property name="hibernate.hbm2ddl.auto" value="update"/>
            <property name="hibernate.show_sql" value="true"/>
            <property name="hibernate.format_sql" value="true"/>
            <property name="hibernate.highlight_sql" value="true"/>
        </properties>
    </persistence-unit>
</persistence>

```

Sustituye `TU_USUARIO` y `TU_PASSWORD` por tus credenciales locales. Crea la base de datos manualmente antes de arrancar: `CREATE DATABASE restaurantes_db;`

Clase utilitaria `JpaUtil`

Implementa esta clase para centralizar la `EntityManagerFactory` :

```
package com.restaurantes.util;

import jakarta.persistence.EntityManager;
import jakarta.persistence.EntityManagerFactory;
import jakarta.persistence.Persistence;

public class JpaUtil {

    private static final EntityManagerFactory emf;

    static {
        try {
            emf = Persistence.createEntityManagerFactory("restaurantesPU");
        } catch (Exception e) {
            throw new ExceptionInInitializerError("Error al iniciar JPA: " + e);
        }
    }

    public static EntityManager createEntityManager() {
        return emf.createEntityManager();
    }

    public static void shutdown() {
        if (emf != null && emf.isOpen()) {
            emf.close();
        }
    }
}
```

Modelo de datos

Enum EstadoReserva

```
package com.restaurantes.modelo;

public enum EstadoReserva {
    PENDIENTE,
    CONFIRMADA,
    CANCELADA,
    COMPLETADA,
    NO_SHOW
}
```

Entidad Restaurante

Representa un local físico de la cadena.

Anotaciones requeridas: `@Entity`, `@Table`, `@Id`, `@GeneratedValue`, `@Column`, `@OneToMany`
Lombok: `@Data`, `@NoArgsConstructor`, `@AllArgsConstructor`

Atributo	Tipo	Descripción
id	Long	Clave primaria, autoincremental
nombre	String	Nombre del restaurante
ciudad	String	Ciudad donde se ubica
direccion	String	Calle y número
telefono	String	Teléfono de contacto
mesas	List<Mesa>	Mesas del restaurante

Relación: Un restaurante puede tener **muchas mesas** (`@OneToMany`).

El atributo `mesas` debe mapearse con `@OneToMany(mappedBy = "restaurante", cascade = CascadeType.ALL, fetch = FetchType.LAZY)`. Excluye este atributo del `toString()` para evitar referencias circulares.

`toString()` esperado:

ID: 1 | La Esquina | Sevilla | C/ Sierpes 12 | 954123456

Entidad Mesa

Representa una mesa concreta dentro de un restaurante.

Anotaciones requeridas: `@Entity`, `@Table`, `@Id`, `@GeneratedValue`, `@Column`, `@ManyToOne`
Lombok: `@Data`, `@NoArgsConstructor`, `@AllArgsConstructor`

Atributo	Tipo	Descripción
id	Long	Clave primaria, autoincremental
numero	int	Número identificador de la mesa
capacidad	int	Número máximo de comensales
terrazza	boolean	Si la mesa está en terraza
restaurante	Restaurante	Restaurante al que pertenece
reservas	List<Reserva>	Reservas asociadas a la mesa

Relaciones:

- Muchas mesas pertenecen a un restaurante → @ManyToOne sobre restaurante .
- Una mesa puede tener muchas reservas → @OneToMany sobre reservas .

Usa @ToString.Exclude en restaurante y reservas para evitar referencias circulares.

toString() esperado:

```
ID: 3 | Mesa 12 | Capacidad: 4 | Terraza: Sí | La Esquina
```

Entidad Cliente

Representa a una persona que realiza reservas.

Anotaciones requeridas: @Entity , @Table , @Id , @GeneratedValue , @Column , @OneToMany

Lombok: @Data , @NoArgsConstructor , @AllArgsConstructor

Atributo	Tipo	Descripción
id	Long	Clave primaria, autoincremental
nombre	String	Nombre y apellidos
email	String	Correo electrónico
telefono	String	Teléfono de contacto
vip	boolean	Si es cliente habitual/VIP
reservas	List<Reserva>	Reservas realizadas por el cliente

Relación: Un cliente puede tener muchas reservas.

toString() esperado:

```
ID: 1 | Marta Pérez | marta@email.com | 600123123 | VIP: Sí
```

Entidad Reserva

Representa una reserva concreta para una mesa.

Anotaciones requeridas: @Entity , @Table , @Id , @GeneratedValue , @Column , @ManyToOne , @Enumerated

Lombok: @Data , @NoArgsConstructor , @AllArgsConstructor

Atributo	Tipo	Descripción
id	Long	Clave primaria, autoincremental
fechaReserva	LocalDate	Día de la reserva
horaReserva	LocalTime	Hora de la reserva
numPersonas	int	Número de comensales
estado	EstadoReserva	Estado de la reserva
importeEstimado	double	Consumo estimado en euros
mesa	Mesa	Mesa reservada
cliente	Cliente	Cliente que realiza la reserva

Relaciones:

- Restaurante → Mesas: @OneToMany
- Mesa → Restaurante: @ManyToOne
- Mesa → Reservas: @OneToMany
- Reserva → Mesa: @ManyToOne
- Cliente → Reservas: @OneToMany
- Reserva → Cliente: @ManyToOne

Usa @Enumerated(EnumType.STRING) para guardar el estado como texto. Marca mesa y cliente con @ToString.Exclude.

La reserva es el objeto que une cliente y mesa, y además guarda datos propios como fecha, hora, estado e importe. Por eso, en este dominio, Reserva es la entidad clave y sustituye a un ManyToMany clásico.

toString() esperado:

```
ID: 7 | 2026-05-12 21:30 | 4 personas | CONFIRMADA | 86.50€ | Marta Pérez | Mesa 12
```

Capa Repository

Implementa una clase repository por cada entidad con, al menos, los siguientes métodos. Todas las operaciones deben gestionar la transacción internamente (begin, commit, rollback en caso de error).

RestauranteRepository

Método	Descripción
<code>guardar(Restaurante r)</code>	Persiste un nuevo restaurante
<code>buscarPorId(Long id)</code>	Devuelve el restaurante con ese id o <code>null</code>
<code>listarTodos()</code>	Devuelve todos los restaurantes
<code>actualizar(Restaurante r)</code>	Actualiza los datos de un restaurante existente
<code>eliminar(Long id)</code>	Elimina un restaurante por su id

MesaRepository

Método	Descripción
<code>guardar(Mesa m)</code>	Persiste una nueva mesa
<code>buscarPorId(Long id)</code>	Devuelve la mesa con ese id o <code>null</code>
<code>listarTodos()</code>	Devuelve todas las mesas
<code>actualizar(Mesa m)</code>	Actualiza los datos de una mesa existente
<code>eliminar(Long id)</code>	Elimina una mesa por su id
<code>buscarPorRestaurante(Long restauranteId)</code>	Devuelve las mesas de un restaurante usando JPQL

ClienteRepository

Método	Descripción
<code>guardar(Cliente c)</code>	Persiste un nuevo cliente
<code>buscarPorId(Long id)</code>	Devuelve el cliente con ese id o <code>null</code>
<code>listarTodos()</code>	Devuelve todos los clientes
<code>actualizar(Cliente c)</code>	Actualiza los datos de un cliente existente
<code>eliminar(Long id)</code>	Elimina un cliente por su id
<code>buscarVip()</code>	Devuelve los clientes VIP usando JPQL

ReservaRepository

Método	Descripción
<code>guardar(Reserva r)</code>	Persiste una nueva reserva
<code>buscarPorId(Long id)</code>	Devuelve la reserva con ese id o <code>null</code>
<code>listarTodos()</code>	Devuelve todas las reservas
<code>actualizar(Reserva r)</code>	Actualiza los datos de una reserva existente
<code>eliminar(Long id)</code>	Elimina una reserva por su id
<code>buscarPorEstado(EstadoReserva estado)</code>	Devuelve las reservas de un estado dado usando JPQL
<code>buscarPorCliente(Long clienteId)</code>	Devuelve todas las reservas de un cliente usando JPQL

En los métodos que usen JPQL, incluye un comentario con la query utilizada.

Servicio `ReservaServicio`

Implementa los siguientes métodos de consulta en `ReservaServicio`, usando los repositorios y/o JPQL directamente. Cada método debe mostrar su resultado por consola con un encabezado claro.

Consulta 1 — Reservas confirmadas ordenadas por fecha

Método: `getReservasConfirmadas()`

Listar todas las reservas con estado `CONFIRMADA`, ordenadas por fecha ascendente.

JPQL: `FROM Reserva r WHERE r.estado = :estado ORDER BY r.fechaReserva ASC`

Consulta 2 — Reservas de un restaurante concreto

Método: `getReservasPorRestaurante(Long restauranteId)`

Mostrar todas las reservas asociadas a un restaurante dado.

JPQL: JOIN sobre `mesa.restaurante`

Consulta 3 — Reservas pendientes para hoy

Método: `getReservasPendientesHoy()`

Obtener las reservas pendientes del día actual.

JPQL: `FROM Reserva r WHERE r.estado = :estado AND r.fechaReserva = :hoy`

Consulta 4 — Recaudación total por restaurante

Método: `getRecaudacionPorRestaurante()`

Calcular la recaudación total estimada de cada restaurante, mostrando el nombre del restaurante y el total, de mayor a menor.

JPQL + *Streams* sobre el resultado.

Consulta 5 — Restaurante con más mesas

Método: `getRestauranteConMasMesas()`

Obtener el restaurante que tenga el mayor número de mesas registradas.

JPQL: `FROM Restaurante ORDER BY SIZE(mesas) DESC + findFirst()`

Consulta 6 — Reservas canceladas o no presentadas

Método: `getReservasProblematicas()`

Listar las reservas con estado `CANCELADA` o `NO_SHOW`, ordenadas por fecha descendente.

JPQL: cláusula `IN (:estados)`

Consulta 7 — Número de reservas por ciudad

Método: `getReservasPorCiudad()`

Mostrar cuántas reservas existen agrupadas por ciudad del restaurante.

JPQL: `SELECT r.mesa.restaurante.ciudad, COUNT(r) FROM Reserva r GROUP BY r.mesa.restaurante.ciudad`

Consulta 8 — Mesas más solicitadas

Método: `getMesasMasSolicitadas()`

Detectar las mesas con más reservas registradas.

JPQL + *Streams* usando el tamaño de la colección o un `COUNT` agrupado.

Consulta 9 — Importe medio por reserva según terraza

Método: `getImporteMedioPorTerraza()`

Calcular el importe medio estimado de las reservas en mesas de terraza y en mesas interiores.

JPQL: `SELECT m.terraza, AVG(r.importeEstimado) FROM Reserva r JOIN r.mesa m GROUP BY m.terraza`

Consulta 10 — Clientes frecuentes

Método: `getClientesFrecuentes(int minimoReservas)`

Obtener los nombres de clientes con al menos `minimoReservas` reservas, sin repeticiones.

JPQL + Streams: `distinct`, `filter`

Datos de prueba

En `Main.java` debes insertar datos de prueba antes de ejecutar las consultas. Como mínimo:

- **2 restaurantes** en ciudades distintas y con estilos distintos.
- **12 mesas** repartidas entre los restaurantes.
- **10 clientes** con mezcla de clientes VIP y no VIP.
- **30 reservas** distribuidas entre mesas y clientes, con mezcla de estados.

Inserta los datos usando los repositorios. No uses SQL directo ni scripts externos.

Ejemplo de salida esperada

=== Consulta 1: Reservas confirmadas ===

ID: 7 | 2026-05-12 21:30 | 4 personas | CONFIRMADA | 86.50€ | Marta Pérez | Mesa 12

ID: 3 | 2026-05-13 20:00 | 2 personas | CONFIRMADA | 42.00€ | Ana López | Mesa 4

=== Consulta 4: Recaudación por restaurante ===

La Esquina → 742.50 €

Sabor Mediterráneo → 615.00 €

Brasa Norte → 488.00 €

=== Consulta 7: Reservas por ciudad ===

Sevilla → 12 reservas

Madrid → 10 reservas

Valencia → 8 reservas

Requisitos técnicos

- Usar **Java 21** o superior con **Hibernate 7** y **MySQL**.
- Proyecto **Maven** con la estructura de paquetes indicada.
- Usar **Lombok** en todas las entidades del modelo.
- Gestionar correctamente las **transacciones** en todos los métodos del repositorio.

- Usar **JPQL** en todas las consultas que lo requieran.
- Activar `hibernate.show_sql=true` para que el SQL generado se muestre en consola.
- Usar `hibernate.hbm2ddl.auto=update` solo para desarrollo.
- El código debe estar **comentado** y ser legible.
- El proyecto debe compilar con `mvn compile exec:java` sin errores.

Entregables

El alumno debe subir a GitHub un repositorio público con el proyecto Maven completo. El repositorio debe contener:

- Todas las clases organizadas en los paquetes indicados.
- `persistence.xml` en `src/main/resources/META-INF/` (sin credenciales reales, usa valores de ejemplo o variables de entorno).
- `Main.java` que inserte los datos de prueba e invoque todos los métodos de `ReservaServicio`.
- `README.md` con: nombre del alumno, descripción de la aplicación, esquema de la BBDD (puedes incluir una captura del diagrama ER) e instrucciones para configurar MySQL y ejecutar el proyecto.

Criterios de evaluación

Criterio	Peso
Correcta definición de entidades y anotaciones JPA	20%
Relaciones <code>@ManyToOne</code> / <code>@OneToMany</code> bien mapeadas	15%
Implementación correcta de los métodos Repository (CRUD + transacciones)	25%
Implementación correcta de las consultas JPQL en <code>ReservaServicio</code>	25%
Claridad, comentarios y estructura del código	10%
README con diagrama ER e instrucciones de ejecución	5%

Las consultas 4, 7 y 9 implican agregaciones JPQL y se valorarán con más peso dentro del bloque de consultas.
